VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY



BÁO CÁO THỰC TẬP

NGÀNH: KHOA HỌC MÁY TÍNH

ĐỀ TÀI:

NGHIÊN CỨU VÀ ỨNG DỤNG AI AGENT TRONG TỰ ĐỘNG HÓA QUY TRÌNH RED TEAMING

Cán bộ hướng dẫn:	Trần Quang Tài
Giảng viên đánh giá:	TS. Lê Khánh Trình
Sinh viên:	Phạm Anh Tuấn
Mã sinh viên:	23021709
Lớp học phần:	QH-2023-I/CQ-I-CS3

Hanoi, 2026

Mục lục

Lời cảm ơn **2**

1 Giới thiệu chung **3**
1.1 Giới thiệu đơn vị thực tập . . . 3
1.2 Giới thiệu công việc . . . 3
1.3 Giới thiệu bài toán . . . 4

2 Yêu cầu bài toán **6**
2.1 Phạm vi và đối tượng . . . 6
2.2 Yêu cầu chức năng và phi chức năng . . . 6
2.3 Phạm vi công việc cá nhân . . . 7
2.4 Tiêu chí đánh giá . . . 8

3 Tóm tắt lý thuyết, giải pháp và cách giải quyết **9**
3.1 Khảo sát các hệ thống liên quan . . . 9
3.1.1 Decepticon – Red team agent đa tác tử . . . 9
3.1.2 Incalmo – Red team đa host với LLM hỗ trợ . . . 11
3.1.3 Tham chiếu từ các hệ thống pentest . . . 12
3.2 Kiến trúc và cách giải quyết đề xuất . . . 12
3.2.1 Kiến trúc tổng thể Raptix . . . 13
3.2.2 Attack Knowledge Graph (AKG) . . . 14
3.2.3 Mô hình khai thác hai pha . . . 16
3.3 Thiết kế benchmark . . . 18
3.4 Liên hệ và so sánh với các cách đã có . . . 20

4 Mô tả phần mềm cài đặt **21**
4.1 Môi trường phát triển và tổ chức mã nguồn . . . 21
4.2 Các artifact phục vụ phát triển . . . 21
4.3 Quy trình chạy một vòng thử nghiệm . . . 22

5 Kết quả đạt được và hướng phát triển **23**
5.1 Kết quả và kỹ năng thu được . . . 23
5.2 Hạn chế và hướng phát triển . . . 24

References **25**

Lời cảm ơn

Trong thời gian thực tập tại Tổng công ty Mạng lưới Viettel (Viettel Networks - VTNet), em đã có cơ hội tiếp cận một bài toán có tính liên ngành giữa trí tuệ nhân tạo và an toàn thông tin. Quá trình thực tập không chỉ giúp em hiểu rõ hơn cách một ý tưởng nghiên cứu về AI Agent được chuyển thành một hệ thống có thể triển khai, kiểm thử và đánh giá trong môi trường thực tế, mà còn giúp em được đào tạo bài bản về quy trình red team thực chiến - từ trinh sát, thiết lập foothold, leo thang đặc quyền, di chuyển ngang cho đến duy trì hiện diện trong mạng mục tiêu.

Em xin chân thành cảm ơn Tổng công ty Mạng lưới Viettel đã tạo điều kiện về môi trường làm việc, tài nguyên nghiên cứu và định hướng kỹ thuật trong suốt thời gian thực tập. Em đặc biệt cảm ơn cán bộ hướng dẫn Trần Quang Tài đã trực tiếp trao đổi, góp ý và giúp em xác định phạm vi bài toán phù hợp, từ khảo sát các hệ thống AI Pentest/Red Team mã nguồn mở đến xây dựng hướng tối ưu và khung benchmark cho hệ thống nội bộ.

Em cũng xin gửi lời cảm ơn chân thành tới các anh chị kỹ sư và chuyên gia tại VTNet, những người đã trực tiếp chia sẻ kiến thức, kinh nghiệm tác chiến thực tế và giới thiệu cho em các giải pháp an toàn thông tin doanh nghiệp tiên tiến đến từ những đối tác hàng đầu như Cisco, cùng nhiều nhà cung cấp khác trên cả nước. Những buổi trao đổi, trình diễn sản phẩm và phân tích kiến trúc phòng thủ doanh nghiệp đó đã giúp em có cái nhìn toàn diện hơn về bối cảnh mà một hệ thống AI Agent Red Team cần phải đối mặt và thích ứng.

Em cũng xin gửi lời cảm ơn tới TS. Lê Khánh Trình và các thầy cô Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội đã trang bị những kiến thức nền tảng về khoa học máy tính và tư duy nghiên cứu, đồng thời hỗ trợ em trong quá trình thực hiện và hoàn thiện báo cáo thực tập này.

1 Giới thiệu chung

1.1 Giới thiệu đơn vị thực tập

Tổng công ty Mạng lưới Viettel (Viettel Networks - VTNet) là đơn vị thuộc Tập đoàn Công nghiệp - Viễn thông Quân đội, gắn với nhiệm vụ quản lý, vận hành và bảo đảm chất lượng hạ tầng mạng viễn thông. Trong các báo cáo chất lượng dịch vụ công khai của Viettel, VTNet được nêu là đơn vị đầu mối về quản lý chất lượng mạng. Đặc thù vận hành hạ tầng quy mô lớn đặt ra yêu cầu cao về tính ổn định, khả năng quan sát, tự động hóa và an toàn thông tin.

Trong bối cảnh các hệ thống mạng và dịch vụ ngày càng phức tạp, hoạt động đánh giá an toàn thông tin không chỉ dừng ở việc quét lỗ hổng đơn lẻ. Một đợt kiểm thử xâm nhập thực tế thường bao gồm nhiều giai đoạn: thu thập thông tin, nhận diện bề mặt tấn công, xác minh lỗ hổng, khai thác trong phạm vi cho phép, nâng quyền, di chuyển ngang, tổng hợp bằng chứng và lập báo cáo. Đây là một chuỗi công việc có nhiều quyết định phụ thuộc ngữ cảnh, vì vậy phù hợp để nghiên cứu khả năng hỗ trợ của mô hình ngôn ngữ lớn và AI Agent.

Trong thời gian thực tập tại VTNet, ngoài việc nghiên cứu và phát triển hệ thống AI Agent, em còn được đào tạo bài bản về quy trình red team thực chiến - từ trinh sát, thiết lập foothold, leo thang đặc quyền, di chuyển ngang cho đến duy trì hiện diện trong mạng mục tiêu. Song song với đó, em có cơ hội được tiếp cận các giải pháp an toàn thông tin doanh nghiệp tiên tiến đến từ các đối tác hàng đầu như Cisco cùng nhiều nhà cung cấp khác trên cả nước, qua đó có cái nhìn toàn diện hơn về bối cảnh mà một hệ thống AI Agent Red Team cần phải đối mặt và thích ứng.

1.2 Giới thiệu công việc

Nội dung thực tập của em tập trung vào việc nghiên cứu và xây dựng định hướng cho một hệ thống AI Agent hỗ trợ tự động hóa quy trình red team trong môi trường được cấp quyền. Khác với pentest truyền thống vốn thường dừng ở việc phát hiện và xác minh lỗ hổng trên một mục tiêu đơn lẻ, red team đòi hỏi khả năng thực thi một chuỗi tấn công hoàn chỉnh trên nhiều host: từ trinh sát, thiết lập foothold, leo thang đặc quyền, di chuyển ngang cho đến duy trì hiện diện và đạt mục tiêu cuối cùng. Đây là bài toán có độ phức tạp cao hơn hẳn về mặt quản lý trạng thái, lập kế hoạch và phối hợp hành động.

Để có cơ sở thiết kế, em đã khảo sát ba hệ thống pentest mã nguồn mở là CyberStrikeAI [1], Strix [2] và Pentest-AI [3] nhằm rút ra các pattern kiến trúc về orchestration, tool execution, sandbox và verification. Tuy nhiên, trọng tâm nghiên cứu của em là hai hệ thống red team tiên tiến: **Decepticon** [4] — framework red team tự động đa tác tử của BitterSecurity, và **Incalmo** [5] — hệ thống red team đa host do LLM hỗ trợ đến từ Carnegie Mellon và Anthropic.

Phân tích chi tiết kiến trúc và điểm mạnh/hạn chế của hai hệ thống này được trình bày ở phần 3.1. Trên cơ sở đó, em xác định hai hướng cải tiến chính cho prototype **Raptix** - hệ thống được em xây dựng kế thừa từ Decepticon, sử dụng kiến trúc đa tác tử và sandbox biệt lập của Decepticon làm nền tảng, đồng thời tiếp thu các bài học về tách planning/execution và quản lý state ngoài context từ Incalmo.

Thứ nhất, **thiết kế lại lớp lưu trữ trạng thái dưới dạng đồ thị tri thức tấn công (Attack Knowledge Graph - AKG)**. Cả Decepticon lẫn Incalmo đều nhận ra hạn chế của việc lưu trữ state phẳng hoặc nhồi toàn bộ vào context LLM; trong khi Decepticon dùng knowledge graph cho findings và Incalmo dùng attack-graph service để off-load knowledge, thì AKG của Raptix hợp nhất tất cả các thực thể (asset, identity, credential, vulnerability, finding, action, evidence) và quan hệ giữa chúng vào một đồ thị duy nhất, đóng vai trò nguồn sự thật cho mọi tác tử. Cách tiếp cận này cho phép truy vấn đường đi tấn công (attack path reasoning), tái sử dụng credential, và loại trừ các nhánh đã thất bại - những khả năng mà knowledge graph của Decepticon và attack-graph service của Incalmo chỉ đáp ứng một phần.

Thứ hai, **phân tách quy trình khai thác thành hai pha với đặc trưng lan truyền khác nhau**: pha Initial Foothold (khai thác theo chiều ngang, breadth-first) tập trung vào độ phủ bề mặt tấn công để giành điểm truy cập đầu tiên; và pha Network Propagation (khai thác theo chiều sâu, depth-first) tập trung vào việc đi sâu vào mạng từ foothold hiện tại để đạt mục tiêu cuối. Sự phân tách này lấy cảm hứng từ cách Incalmo mô hình hóa các declarative task theo kill chain [5] và cách Decepticon phân chia sub-agent theo pha (recon, exploit, post-exploit) [4], nhưng bổ sung cơ chế chuyển pha tường minh dựa trên trạng thái của AKG - điều mà cả hai hệ thống tham chiếu đều chưa có.

Song song với thiết kế hệ thống, em xây dựng khung benchmark để tránh việc tối ưu theo cảm tính, dựa trên ba trục Effectiveness, Efficiency và Reliability. Mỗi thay đổi kiến trúc cần được đánh giá không chỉ bằng solve rate mà còn bằng token cost, thời gian thực thi và độ ổn định qua nhiều lần chạy, nhằm đảm bảo các cải tiến thực sự mang lại giá trị thay vì chỉ dịch chuyển chi phí từ trục này sang trục khác.

1.3 Giới thiệu bài toán

Bài toán tổng quát là xây dựng một tác nhân AI có khả năng nhận mục tiêu kiểm thử ở mức cao, tự lập kế hoạch, lựa chọn và gọi các công cụ an toàn thông tin, đọc kết quả, cập nhật hiểu biết về mục tiêu và quyết định bước tiếp theo. Khác với chatbot chỉ trả lời hướng dẫn, hệ thống agentic phải duy trì vòng lặp hành động - quan sát - suy luận - cập nhật trạng thái trong thời gian dài.

Khó khăn chính xuất phát từ tính không xác định của môi trường red team. Một lệnh có thể thất bại vì công cụ thiếu, timeout, đầu ra quá dài, thông tin xác thực không đúng hoặc mục tiêu thay đổi trạng thái. Nếu agent chỉ dựa vào context hội thoại, sau nhiều bước nó dễ quên bằng chứng cũ, lặp lại thao tác hoặc đưa ra giả định không còn đúng. Vì vậy, bài toán cần một lớp trạng thái bền vững và có cấu trúc thay vì coi toàn bộ lịch sử chat là "bộ nhớ".

Mặt khác, kết quả red team không thể chỉ là một nhận định ngôn ngữ. Mỗi finding cần gắn với asset, thao tác xác minh, bằng chứng đầu ra, tác động và mức độ tin cậy. Đây là điểm mà Pentest-AI nhấn mạnh thông qua cơ chế kiểm chứng lại finding bằng các oracle [3], trong khi CyberStrikeAI chú trọng workspace có thể audit, knowledge và attack-chain [1], còn Strix chú trọng sandbox, skill và graph of agents [2]. Với các hệ thống red team, Decepticon nhấn mạnh vào việc lưu findings vào knowledge graph và vận hành trong sandbox biệt lập [4], còn Incalmo tách biệt planning và execution với các auxiliary services để duy trì state ngoài context LLM [5]. Các hướng này bổ sung cho nhau và là cơ sở cho giải pháp được đề xuất.

Trong phạm vi thực tập, mục tiêu không phải thay thế hoàn toàn red teamer. Hệ thống được định hướng như một nền tảng tự động hóa có kiểm soát: con người xác định scope và mục tiêu, agent chịu trách nhiệm lập kế hoạch và thực thi trong ranh giới đó, còn mọi thao tác quan trọng phải để lại bằng chứng có thể xem lại. Môi trường đánh giá được giới hạn trong lab/benchmark được cấp quyền; không áp dụng cho mục tiêu ngoài phạm vi.

2 Yêu cầu bài toán

2.1 Phạm vi và đối tượng

Đối tượng nghiên cứu của đề tài là hệ thống AI Agent Red Team tự động, hoạt động theo mô hình black-box hoặc gray-box trong môi trường được cấp quyền. Agent không được giả định có quyền truy cập mã nguồn, tài liệu thiết kế hay thông tin nội bộ của mục tiêu nếu benchmark không cung cấp. Khác với các hệ thống pentest chỉ tập trung vào một mục tiêu đơn lẻ, hệ thống được nghiên cứu phải có khả năng thực thi chuỗi tấn công hoàn chỉnh trên nhiều host, từ giai đoạn trinh sát ban đầu cho đến khi đạt mục tiêu cuối cùng.

Các capability chính được xác định cho hệ thống bao gồm: khai thác web (web exploitation); khai thác dịch vụ hạ tầng Linux và giao thức mạng; chiếm đoạt nhiều host (multi-host compromise); leo thang đặc quyền (privilege escalation); pivoting qua các host trung gian; di chuyển ngang (lateral movement); tấn công Windows/Active Directory; và các chiến dịch red team dài hạn (long-horizon operations). Việc tách capability thành các nhóm riêng biệt giúp khung benchmark không bị lệch về một nhóm lỗ hổng dễ đo như web injection, đồng thời phản ánh chính xác hơn yêu cầu của một hệ thống red team tổng quát.

Do đặc thù của đề tài, phạm vi nghiên cứu được giới hạn trong môi trường lab và benchmark được cấp quyền. Mọi hoạt động khai thác chỉ diễn ra trên các target cô lập, có scope và objective được xác định trước, tuân thủ nguyên tắc không ảnh hưởng đến hệ thống ngoài phạm vi. Hệ thống không được thiết kế để hoạt động trong môi trường sản xuất thực tế nếu chưa có sự phê duyệt tường minh từ các bên liên quan.

2.2 Yêu cầu chức năng và phi chức năng

Yêu cầu chức năng

Hệ thống Raptix cần đáp ứng các yêu cầu chức năng sau:

- Nhận mission (mục tiêu, scope, policy) và duy trì ranh giới mục tiêu trong toàn bộ phiên làm việc.
- Lập kế hoạch và phân rã nhiệm vụ theo hai pha: **Initial Foothold** (khai thác theo chiều ngang) và **Network Propagation** (khai thác theo chiều sâu). Cơ chế chuyển pha dựa trên trạng thái của Attack Knowledge Graph (AKG).
- Gọi công cụ qua interface thống nhất, quản lý timeout, lỗi, output lớn và trạng thái thực thi; hỗ trợ các phiên tương tác dài như `msfconsole` hay `sliver-client`.
- Lưu trữ có cấu trúc các thực thể asset, identity, credential, vulnerability, finding, action và evidence vào AKG; biểu diễn quan hệ giữa chúng dưới dạng cạnh có nhãn và có hướng.

- Truy vấn AKG để suy luận attack path: tìm đường đi từ foothold hiện tại tới mục tiêu cuối, phát hiện credential reuse path, privilege escalation path, và loại trừ các nhánh đã thất bại.
- Tái sử dụng kiến thức ổn định (playbook, tool usage, remediation pattern) nhưng không biến suy đoán chưa kiểm chứng thành fact.
- Sinh báo cáo cuối cùng dựa trên bằng chứng; chỉ kết luận khi mức xác minh của finding đáp ứng tiêu chí (candidate, validated, verified, reported).

Yêu cầu phi chức năng

- **Auditability:** có thể truy ngược mọi finding về tool call và bằng chứng tương ứng thông qua event stream của AKG.
- **Reliability:** có khả năng tiếp tục khi tool lỗi, timeout hoặc agent con thất bại; state được phục hồi từ event stream mà không mất mát.
- **Efficiency:** kiểm soát token, model calls, tool calls, thời gian thực thi và coordination overhead; hạn chế việc nhồi toàn bộ state vào context LLM.
- **Extensibility:** có thể thêm tool, skill, agent hoặc benchmark mới mà không phá vỡ contract hiện có.
- **Safety & scope control:** mọi hoạt động thực thi phải nằm trong phạm vi đã được phê duyệt; scope guard được kiểm tra ở tầng tool executor trước khi chạy lệnh.

2.3 Phạm vi công việc cá nhân

Trong bài toán lớn về tự động hóa red team bằng AI Agent, phần việc của em tập trung vào lớp nghiên cứu kiến trúc và khung đánh giá, đồng thời chuẩn bị nền tảng để triển khai các thay đổi vào prototype Raptix. Em không đặt mục tiêu ghép càng nhiều công cụ càng tốt, mà ưu tiên xác định những cơ chế làm hệ thống chạy đúng trong mission dài: quản lý state trên đồ thị, điều phối agent theo hai pha, governance cho tool, kiểm chứng bằng chứng và đo non-regression.

Cụ thể, các hạng mục công việc chính bao gồm:

- Khảo sát và phân tích Decepticon cùng Incalmo để rút ra các pattern kiến trúc về orchestration, sandbox, tách planning/execution và quản lý state ngoài context.
- Thiết kế Attack Knowledge Graph (AKG) với schema cho node và edge, các invariant (schema, provenance, scope) và cơ chế versioning.
- Thiết kế mô hình khai thác hai pha (Initial Foothold, Network Propagation), tiêu chí chuyển pha và cơ chế đồng bộ giữa hai pha.

- Xây dựng quy trình benchmark theo ba trục Effectiveness, Efficiency và Reliability, với baseline được pin theo commit, model, provider và toolset.

Kết quả của giai đoạn này chưa nhằm khẳng định Raptix vượt các hệ thống tham chiếu về solve rate, mà tạo "đường ray" cho quá trình phát triển tiếp theo: biết cần thay đổi ở đâu, đo bằng gì và dữ liệu nào phải được giữ để kết luận.

2.4 Tiêu chí đánh giá

Khung đánh giá của đề tài sử dụng ba trục chính nhằm tránh việc tối ưu theo cảm tính hoặc chỉ tập trung vào một khía cạnh duy nhất:

- **Effectiveness:** đo hệ thống có thực sự phát hiện, xác minh và hoàn thành mục tiêu tấn công hợp lệ hay không. Các chỉ số bao gồm task success rate, số verified findings, số flag hoặc objective đạt được, và mức độ hoàn thành attack-chain.
- **Efficiency:** đo lượng tài nguyên cần dùng để đạt kết quả. Các chỉ số bao gồm token tiêu thụ, số model calls, số tool calls, wall-clock time, số vòng retry và coordination overhead giữa các agent.
- **Reliability:** đo khả năng lặp lại kết quả qua nhiều lần chạy và khả năng sống sót trước lỗi. Các chỉ số bao gồm tỷ lệ thành công qua nhiều run, variance, khả năng recovery sau timeout hoặc failure, và tính nhất quán của state.

Để so sánh công bằng giữa các phiên bản của Raptix và với baseline, mọi yếu tố môi trường cần được cố định trước khi bắt đầu một vòng tối ưu: commit của mã nguồn, model và provider LLM, toolset, cấu hình benchmark và giới hạn tài nguyên. Sau mỗi nhóm thay đổi, hệ thống được chạy lại trên cùng điều kiện; nếu tăng solve rate nhưng chi phí hoặc độ bất ổn tăng quá lớn, thay đổi đó không được xem là tối ưu toàn diện.

Ngoài ba trục chính, do đặc thù của mô hình khai thác hai pha, benchmark cần ghi thêm một số chỉ số bổ trợ: thời điểm chuyển pha Initial Foothold sang Network Propagation, số attack path đã được thử trong AKG, và tỷ lệ credential được tái sử dụng thành công trên nhiều host. Những chỉ số này giúp đánh giá đúng đắn vai trò của AKG và cơ chế chuyển pha trong toàn bộ vòng đời mission.

3 Tóm tắt lý thuyết, giải pháp và cách giải quyết

Trong báo cáo này, **AI Agent** được hiểu là hệ thống có LLM làm thành phần suy luận trung tâm, bao quanh bởi ba lớp: state (trạng thái hệ thống về mục tiêu), tool interfaces (giao diện gọi công cụ) và control loop (vòng lặp observe–plan–act–update). Chất lượng của hệ thống phụ thuộc lớn vào cách thiết kế runtime, không chỉ vào bản thân LLM. Phần này trình bày khảo sát các hệ thống liên quan và kiến trúc đề xuất.

3.1 Khảo sát các hệ thống liên quan

Để có cơ sở thiết kế cho prototype Raptix, em đã khảo sát hai nhóm hệ thống: (i) các hệ thống red team tự động tiên tiến, đóng vai trò là nguồn cảm hứng chính; và (ii) các hệ thống pentest mã nguồn mở, đóng vai trò tham chiếu cho các pattern kiến trúc về orchestration, tool execution và verification. Trong đó, Decepticon và Incalmo là hai đối tượng nghiên cứu trọng tâm, vì chúng đại diện cho hai cách tiếp cận khác nhau nhưng bổ sung cho nhau trong việc giải quyết bài toán red team đa host.

3.1.1 Decepticon – Red team agent đa tác tử

Decepticon [4] là một framework red team tự động mã nguồn mở được phát triển bởi BitterSecurity, định vị là một "red team agent chuyên nghiệp" thay vì một công cụ pentest thông thường. Decepticon thực thi các chuỗi tấn công thực tế bao gồm trinh sát, khai thác, leo thang đặc quyền, di chuyển ngang và C2 — theo cách mà một kẻ tấn công thực sự sẽ làm, chứ không phải theo cách một scanner hoạt động. Điểm khác biệt căn bản của Decepticon so với các công cụ "AI + hacking" khác nằm ở kỷ luật vận hành: trước khi gửi bất kỳ gói tin nào, Decepticon tạo ra một gói engagement hoàn chỉnh bao gồm Rules of Engagement (RoE), Concept of Operations (CONOPS), Deconfliction Plan và OPPLAN với ánh xạ MITRE ATT&CK.

Kiến trúc đa tác tử theo pha kill chain Decepticon tổ chức hệ thống của mình như một **multi-agent stack** được phân chia theo từng pha của kill chain, bao gồm ba orchestrator chính và một dàn sub-agent chuyên biệt:

- **Decepticon:** orchestrator trung tâm của red team, có nhiệm vụ đọc OPPLAN, điều phối các sub-agent và theo dõi chuyển đổi trạng thái của từng objective. Bản thân orchestrator này không có tool nào — mọi công việc tấn công đều được thực hiện thông qua cơ chế delegation đến các sub-agent.
- **Vulnresearch:** orchestrator phụ trách nghiên cứu lỗ hổng, vận hành pipeline năm giai đoạn (scanner → detector → verifier → patcher → exploiter) với state được truyền giữa các giai đoạn *độc quyền thông qua knowledge graph*.

- **Soundwave:** orchestrator phụ trách tương tác với người vận hành để xây dựng OPPLAN và các tài liệu engagement khi chưa có sẵn.
- **Sub-agent chuyên biệt:** dưới quyền Decepticon có một dàn sub-agent như `recon`, `exploit`, `postexploit`, `ad_operator` (Active Directory), `cloud_hunter`, `phisher`, `contract_audi` (smart contract), `reverser` (phân tích binary), và nhiều agent khác cho các lĩnh vực như IoT, ICS, wireless, mobile và OSINT.

Mỗi sub-agent được thiết kế để khởi động với một **context window hoàn toàn mới** cho từng mục tiêu, tránh tích tụ "nhiễu" và suy giảm khả năng suy luận của LLM. Các phát hiện (findings) được lưu trữ trên đĩa (thư mục `workspace/`) và trong knowledge graph, *không* lưu trong bộ nhớ của agent. Đây là một quyết định thiết kế quan trọng, cho phép Decepticon duy trì tính nhất quán của state qua các phiên làm việc dài mà không bị giới hạn bởi context window của LLM.

Knowledge graph và attack chain planning Decepticon sử dụng **Neo4j** làm persistent attack graph. Mọi host, service, vulnerability, credential và finding được phát hiện trong một engagement đều trở thành một **typed node** với các **typed relationship** tương ứng. Đồ thị này không chỉ là nơi lưu trữ thụ động mà còn là công cụ suy luận chủ động:

- **Truy vấn đường đi:** công cụ `kg_query_paths(start_id, end_id)` tìm tất cả các đường đi giữa hai node, phục vụ việc khám phá attack chain.
- **Lập kế hoạch attack chain:** công cụ `plan_attack_chains()` sinh ra các multi-hop exploit path được xếp hạng bằng thuật toán weighted shortest-path. Trọng số của cạnh được dùng để biểu diễn mức độ dễ khai thác (lower = easier exploitation).
- **Đánh giá critical path:** công cụ `critical_path_score(chain)` cho phép đánh giá mức độ quan trọng của một attack chain cụ thể.
- **Cypher-native path search:** planner sử dụng APOC (Awesome Procedures on Cypher) của Neo4j để tìm đường đi ngắn nhất có trọng số từ các `Entrypoint` node đến các `CrownJewel` node.

Đáng chú ý, Neo4j container được thiết kế **dual-homed** trên cả hai mạng `sandbox-net` và `decepticon-net`, cho phép orchestrator trên management plane đọc được các finding do agent ghi từ bên trong sandbox. Đây là một giải pháp kiến trúc cho phép tách biệt mặt phẳng thực thi và mặt phẳng quản lý mà vẫn duy trì được state chia sẻ.

Sandbox và hạn chế Decepticon thực thi lệnh trong sandbox Kali Linux biệt lập qua phiên tmux liên tục, cho phép tương tác với `msfconsole`, `sliver-client`, `evil-winrm` một cách tự nhiên. Neo4j được cấu hình dual-homed để agent trong sandbox ghi findings và orchestrator trên management plane đọc được.

Hạn chế chính là **chưa có cơ chế chuyển pha tường minh**: việc chuyển giữa các giai đoạn kill chain dựa vào quyết định của orchestrator đọc OPPLAN, không dựa trên trạng thái thực tế của knowledge graph. Knowledge graph cũng chưa được tận dụng cho suy luận attack vector thời gian thực, mà chỉ dùng cho path planning tĩnh.

3.1.2 Incalmo – Red team đa host với LLM hỗ trợ

Incalmo là một hệ thống red team đa host do LLM hỗ trợ, được phát triển bởi nhóm nghiên cứu tại Đại học Carnegie Mellon (CMU) và Anthropic. Khác với Decepticon (là một framework hoàn chỉnh), Incalmo là một *high-level abstraction layer* đóng vai trò trung gian giữa LLM và môi trường tấn công. Triết lý thiết kế của Incalmo xuất phát từ một quan sát quan trọng: các hệ thống LLM-assisted offense hiện đại (như PentestGPT, CyberSecEval3) với các LLM hàng đầu (Sonnet 4, Gemini 2.5 Pro) **không thể** tự động thực thi các bài toán red team đa host nếu không có sự hỗ trợ của abstraction layer phù hợp.

Tách biệt planning và execution Đóng góp cốt lõi của Incalmo là việc **tách biệt rõ ràng lớp planning và lớp execution**. Thay vì để một LLM vừa lập kế hoạch vừa sinh ra các lệnh shell cấp thấp, Incalmo chia bài toán thành hai lớp:

- **Lớp planning (LLM-assisted):** quyết định "làm gì" ở mức trừu tượng cao. Planner không sinh ra lệnh shell mà chỉ chọn từ một **vocabulary hữu hạn các high-level attack verbs** như: `scan network`, `move laterally`, `gain persistence`, `escalate privileges`, `exfiltrate data`, `discover local information`.
- **Lớp execution (domain-specific task agents):** quyết định "thực thi như thế nào". Khi planner chọn một verb, một **translation layer** sẽ ánh xạ verb đó thành implementation cấp thấp phù hợp, gọi đến task agent chuyên biệt (port scanner, credential dumper, lateral movement toolkit, v.v.).

Cách tiếp cận này mang lại hai lợi ích quan trọng. Thứ nhất, nó **thu hẹp không gian quyết định** của LLM, giúp model tập trung vào strategic reasoning thay vì phải nhớ cú pháp của hàng trăm công cụ khác nhau. Thứ hai, nó cho phép **thay thế hoặc nâng cấp task agent mà không cần huấn luyện lại planner** — một tính chất quan trọng cho khả năng bảo trì và mở rộng hệ thống.

Auxiliary services, kết quả và hạn chế Để giải quyết context bloat, Incalmo đưa vào ba auxiliary services: *environment state service* (host, service, credential đã thu được), *attack graph service* (đề xuất attack path tiềm năng) và *C2 service* (quản lý kênh liên lạc với host đã bị chiếm). Nhờ đó, planner chỉ truy vấn những gì cần thiết tại thời điểm quyết định, không cần nhồi toàn bộ lịch sử vào context.

Trên benchmark MHBench (40 mạng mô phỏng 22–50 host), Incalmo chiếm được critical asset trong **37/40** môi trường, so với 3/40 của SOTA khác. Thí nghiệm ablation Incalmo-WAP (bỏ action planner) cho thấy **tất cả 6 LLM đều không thể hoàn thành**, khẳng định vai trò thiết yếu của planning layer tách biệt.

Hạn chế: Incalmo chưa có verifier độc lập (kết quả dựa trên output của task agent) và attack graph service chưa hỗ trợ audit/replay theo event stream.

3.1.3 Tham chiếu từ các hệ thống pentest

Ba hệ thống pentest mã nguồn mở bổ sung thêm pattern kiến trúc: **CyberStrikeAI** [1] (MCP/tool registry, audit-oriented workspace), **Strix** [2] (sandbox + skill + graph of agents) và **Pentest-AI** [3] (verification-centric lifecycle với oracle, proof replay và scope guard). Bài học quan trọng nhất từ Pentest-AI là tách "suy luận của LLM" khỏi "điều kiện xác nhận kết quả" — điểm yếu mà cả Decepticon lẫn Incalmo đều chưa giải quyết triệt để. Decepticon [4] (tự báo cáo XBOW 98.08%, knowledge graph làm state store) và Incalmo [5] (MHBench 37/40, tách planning/execution) cùng nhau cung cấp nền tảng cho hai đóng góp cốt lõi của Raptix: AKG hợp nhất và mô hình khai thác hai pha.

3.2 Kiến trúc và cách giải quyết đề xuất

Sau khi phân tích các hệ thống liên quan ở phần 3.1, em chọn hướng tiếp cận **evolutionary redesign** cho prototype Raptix: kế thừa Decepticon làm nền tảng vì kiến trúc multi-agent theo kill chain và sandbox biệt lập của Decepticon đã được kiểm chứng [4], đồng thời tiếp thu các bài học quan trọng từ Incalmo [5] về tách planning/execution và quản lý state ngoài context. Cách tiếp cận này có rủi ro thấp hơn so với việc viết lại từ đầu, và cho phép sử dụng benchmark làm hàng rào non-regression trong suốt quá trình phát triển.

Đóng góp thiết kế của Raptix tập trung vào hai điểm khác biệt cốt lõi so với Decepticon gốc:

- **Attack Knowledge Graph (AKG):** thay thế cách tổ chức knowledge graph hiện tại của Decepticon bằng một đồ thị *hợp nhất* chứa tất cả các thực thể (asset, identity, credential, vulnerability, finding, action, evidence) và quan hệ giữa chúng, đóng vai trò nguồn sự thật duy nhất cho mọi tác tử.
- **Mô hình khai thác hai pha:** phân tách quy trình khai thác thành pha Initial Foothold (khai thác theo chiều ngang, breadth-first) và pha Network Propagation (khai thác theo chiều sâu, depth-first), với cơ chế chuyển pha tường minh dựa trên trạng thái của AKG.

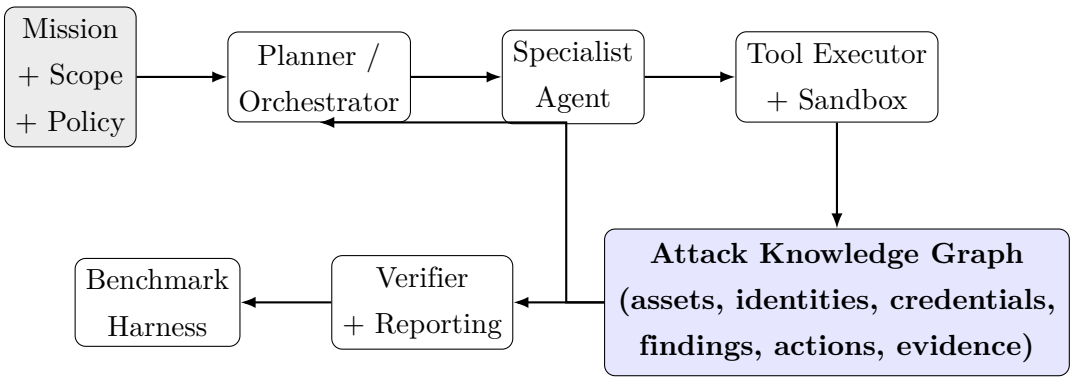
Ba phần con của mục này lần lượt trình bày: kiến trúc tổng thể Raptix (3.2.1), thiết kế chi tiết AKG (3.2.2) và mô hình khai thác hai pha (3.2.3).

3.2.1 Kiến trúc tổng thể Raptix

Raptix được tổ chức thành năm lớp chức năng có ranh giới rõ ràng, giao tiếp với nhau qua các interface được định nghĩa tường minh. Việc tách lớp này giúp hệ thống dễ kiểm thử, dễ mở rộng và cho phép thay thế từng thành phần (ví dụ: đổi LLM provider, đổi graph backend) mà không ảnh hưởng đến các lớp khác.

Năm lớp chức năng

- **Lớp Mission và Policy:** chịu trách nhiệm nhận mission từ operator, kiểm tra scope và policy, nạp thông tin ban đầu vào AKG, và duy trì ranh giới mục tiêu trong suốt phiên làm việc. Đây là lớp bảo vệ đầu tiên, đảm bảo mọi hành động tiếp theo đều nằm trong phạm vi được cấp quyền.
- **Lớp Planner và Orchestrator:** thực hiện phân rã nhiệm vụ, chọn chiến lược khai thác theo pha hiện tại, và điều phối các sub-agent chuyên biệt. Planner chỉ thao tác ở mức trừu tượng cao — chọn *làm gì* — chứ không sinh ra lệnh shell cấp thấp.
- **Lớp Specialist Agent:** gồm các sub-agent chuyên biệt theo domain (recon, web, AD, cloud, post-exploit...). Mỗi agent chỉ được khởi tạo khi cần, với context window sạch và nhiệm vụ được định nghĩa rõ ràng. Agent *không* giữ state riêng ngoài context tạm thời — mọi state đều đọc/ghi qua AKG.
- **Lớp Tool Executor và Sandbox:** chịu trách nhiệm thực thi lệnh trong sandbox Kali Linux biệt lập. Mọi tool call phải đi qua scope guard và audit logger trước khi được đưa xuống sandbox. Tầng này hỗ trợ các phiên tương tác dài thông qua tmux sessions, cho phép làm việc với các công cụ như `msfconsole` hay `sliver-client`.
- **Lớp Knowledge và Verification:** bao gồm AKG (lưu trữ tri thức tấn công), event stream (audit và replay), verifier (xác minh finding độc lập), và reporting engine (sinh báo cáo cuối cùng dựa trên bằng chứng).



Hình 1: Luồng xử lý logic của Raptix với AKG ở trung tâm

Sơ đồ luồng xử lý

Nguyên tắc thiết kế Sáu nguyên tắc thiết kế định hình toàn bộ kiến trúc Raptix:

- **Scope là dữ liệu hạng nhất:** mọi tool call phải được kiểm tra theo scope và policy trước khi chạy. Scope không phải là ràng buộc ngầm trong prompt mà là dữ liệu có cấu trúc được lưu trong AKG và được enforce ở tầng executor.
- **LLM context là cache làm việc, không phải nguồn sự thật:** nguồn sự thật duy nhất là AKG. Điều này cho phép agent khởi động lại, thay đổi model, hoặc chạy song song mà vẫn giữ tính nhất quán của state.
- **Tool execution có định danh và vòng đời rõ ràng:** mỗi tool call có execution_id, trạng thái (pending, running, completed, failed, cancelled), timeout và output cap, cho phép mission dài có thể resume sau sự cố.
- **Agent specialization chỉ dùng khi có lợi ích rõ:** multi-agent chỉ được kích hoạt khi việc phân tách nhiệm vụ giảm complexity hoặc tăng parallelism đủ để bù đắp coordination overhead. Không mặc định multi-agent cho mọi task.
- **Finding có lifecycle chặt chẽ:** mỗi finding đi qua các trạng thái candidate → validated → verified → reported. Không sinh finding "chắc chắn" chỉ từ suy luận của LLM.
- **Graph là nguồn sự thật duy nhất:** mọi agent đọc/ghi qua AKG; không agent nào giữ state riêng ngoài context tạm thời. Điều này đảm bảo tính nhất quán khi nhiều agent chạy song song.

3.2.2 Attack Knowledge Graph (AKG)

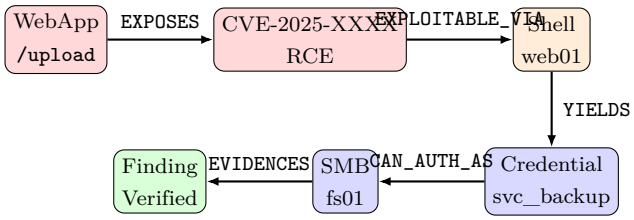
Attack Knowledge Graph (AKG) là đóng góp thiết kế cốt lõi của Raptix. Đây là một đồ thị có hướng, có nhãn, được thiết kế để lưu trữ *tất cả* các thực thể và quan hệ mà một red team agent cần trong suốt mission, thay thế cho mô hình state phẳng hoặc nhồi toàn bộ vào context LLM.

Các loại node trong AKG AKG phân loại node theo bảy nhóm chính, tương ứng với các thực thể mà một red teamer thao tác trong thực tế:

Bảng 1: Các loại node trong Attack Knowledge Graph

Nhóm node	**Ví dụ**	**Vai trò**
Asset	Host, Port, Service, Endpoint, Application	Mô tả bề mặt tấn công đã quan sát được.
Identity	User, Account, Group, Role	Đối tượng xác thực và phân quyền.
Credential	Password, NTLM hash, Kerberos ticket, SSH key, Session cookie	Vật liệu xác thực thu được trong quá trình khai thác.
Vulnerability	CVE, Misconfiguration, Logic flaw	Lỗ hổng hoặc điểm yếu đã nhận diện.
Finding	Candidate, Validated, Verified, Reported	Kết luận về một vấn đề an toàn thông tin, có lifecycle bốn bước rõ ràng.
Action	Tool call, Command, Exploit attempt	Bản ghi thao tác đã thực thi, phục vụ audit và replay.
Evidence	stdout, HTTP response, Screenshot, File artifact	Dữ liệu thô hỗ trợ cho finding hoặc state.

Các loại cạnh trong AKG Quan hệ giữa các node được biểu diễn bằng các cạnh có nhãn, có hướng và trọng số (chi phí, rủi ro OPSEC, mức tin cậy). Mười ba cạnh chính: `HOSTS`, `RUNS_ON`, `EXPOSES`, `HAS_CREDENTIAL`, `CAN_AUTH_AS`, `MEMBER_OF`, `HAS_PRIVILEGE`, `EXPLOITABLE_VIA`, `YIELDS` (Action → Credential), `LEADS_TO` (Action → Finding), `EVIDENCES`, `CONNECTS_TO`, `PIVOT_VIA`.



Hình 2: Ví dụ một đoạn AKG sau pha Initial Foothold

Ví dụ một đoạn AKG

Cơ chế cập nhật và tính nhất quán AKG ghi event theo mô hình **append-only** (bất biến) và state là ảnh chụp tái tạo được từ event stream (versioned). Kết hợp hai nguyên tắc này đảm bảo: auditability (truy vết finding về event gốc), replayability (dựng lại toàn mission để debug), concurrency (ghi song song qua transaction theo node) và recovery (phục hồi sau crash mà không mất dữ liệu).

Ba invariant của AKG Để đảm bảo tính đúng đắn của đồ thị trong suốt mission, AKG được bảo vệ bởi ba invariant:

- **Schema invariant:** mọi node và edge phải tuân theo schema đã định nghĩa; không cho phép node hoặc edge "mồ côi" (không có node cha).
- **Provenance invariant:** mọi finding phải truy vết được về evidence và action đã tạo ra nó. Finding không có evidence bị coi là chưa xác minh.
- **Scope invariant:** mọi node asset phải nằm trong scope đã được phê duyệt; node ngoài scope bị từ chối ngay tại thời điểm ghi. Đây là lớp bảo vệ thứ hai, độc lập với scope guard ở tầng tool executor.

Lựa chọn backend AKG không phụ thuộc backend cụ thể. Trong giai đoạn thực tập, Neo4j được chọn làm backend chính vì tương thích với Decepticon và hỗ trợ Cypher-native path search với APOC; Memgraph (tương thích Cypher) và graph store nhúng (NetworkX/igraph) là lựa chọn thay thế cho môi trường streaming hoặc lab nhẹ.

3.2.3 Mô hình khai thác hai pha

Điểm khác biệt quan trọng thứ hai của Raptix so với Decepticon gốc là việc phân tách quy trình khai thác thành hai pha với *chiến lược lan truyền* khác nhau. Sự tách biệt này phản ánh đúng thực tế tác chiến red team: giai đoạn đầu cần *độ phủ* (breadth) để tìm điểm vào, giai đoạn sau cần *độ sâu* (depth) để đạt mục tiêu cuối.

Pha 1 - Initial Foothold Agent quét bề mặt tấn công theo chiều ngang (breadth-first, song song hóa cao) để liệt kê asset, service và điểm vào tiềm năng. AKG ghi các node Asset, Service, Vulnerability (candidate), Credential qua cạnh `EXPOSES`, `HOSTS`, `RUNS_ON`. Pha kết thúc khi có ít nhất một foothold được xác minh và bề mặt đủ để lập kế hoạch pha sau.

Pha 2 - Network Propagation Từ foothold, agent chuyển sang depth-first (song song hóa thấp, tuần tự theo attack path) để leo thang đặc quyền, thu credential và di chuyển ngang. AKG ghi node Credential, Identity (đặc quyền được biểu diễn qua cạnh `HAS_PRIVILEGE`) qua cạnh `CAN_AUTH_AS`, `MEMBER_OF`, `HAS_PRIVILEGE`, `PIVOT_VIA`. Pha kết thúc khi đạt mục tiêu cuối hoặc hết ngân sách tài nguyên.

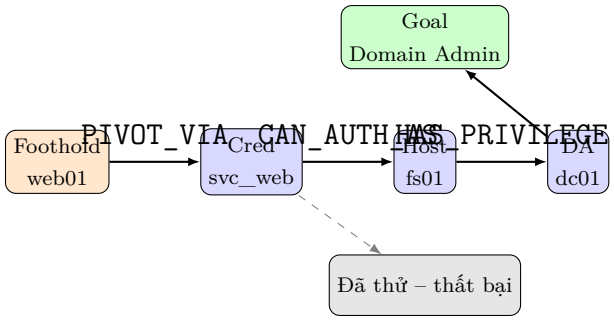
Cơ chế chuyển pha Điểm cốt lõi của mô hình hai pha là **cơ chế chuyển pha tường minh dựa trên trạng thái của AKG**. Cụ thể, Raptix duy trì một tập các điều kiện chuyển pha được đánh giá liên tục sau mỗi bước hành động:

- **Điều kiện chuyển từ pha 1 sang pha 2:** AKG chứa ít nhất một node Finding thuộc loại foothold đã đạt trạng thái `verified`, đồng thời bề mặt tấn công đã được liệt kê tới mức đủ để planner có thể đề xuất ít nhất một attack path khả thi tới mục tiêu cuối.

- **Điều kiện quay lại pha 1:** trong pha 2, nếu agent gặp một subnet mới chưa được liệt kê (thể hiện bằng node Asset không có cạnh `HOSTS` hoặc `CONNECTS_TO` tới các node đã biết), planner có thể tạm thời chuyển sang chế độ breadth-first để mở rộng bề mặt, sau đó quay lại depth-first.
- **Điều kiện kết thúc:** mục tiêu cuối đã đạt được, hoặc ngân sách token/thời gian đã hết, hoặc operator ra lệnh dừng.

Mọi chuyển pha đều được ghi lại trong AKG như một event có timestamp, lý do chuyển pha và trạng thái đồ thị tại thời điểm chuyển. Điều này cho phép audit và replay toàn bộ quá trình chuyển pha.

Tận dụng AKG cho attack vector reasoning Trong pha 2, AKG đóng vai trò công cụ suy luận chủ động: (i) truy vấn đường đi từ foothold tới mục tiêu bằng Dijkstra/A*; (ii) phát hiện credential reuse path qua `HAS_CREDENTIAL` và `CAN_AUTH_AS`; (iii) phát hiện privilege escalation path qua `MEMBER_OF` và `HAS_PRIVILEGE`; (iv) loại trừ nhánh đã thất bại (đánh dấu `status: failed`); (v) ưu tiên hóa theo chi phí và rủi ro OPSEC.



Hình 3: Ví dụ attack path depth-first trong pha Network Propagation; nhánh thất bại được đánh dấu để tránh lặp lại

Bảng 2: So sánh đặc trưng của hai pha khai thác

Tiêu chí	**Initial Foothold**	**Network Propagation**
Chiến lược	Breadth-first, ưu tiên độ phủ	Depth-first, ưu tiên đường tới mục tiêu
Mục tiêu	Điểm vào đầu tiên + bản đồ bề mặt	Mục tiêu cuối (DA, data, flag)
Song song hóa	Cao, nhiều target cùng lúc	Thấp, phụ thuộc tuần tự theo attack path
Vai trò AKG	Ghi nhận asset, service, vuln	Ghi nhận credential, privilege, pivot
Rủi ro chính	Bỏ sót bề mặt tấn công	Đi sai nhánh, tốn tài nguyên

So sánh hai pha

3.3 Thiết kế benchmark

Benchmark của Raptix được thiết kế để phục vụ hai mục đích song song: (i) so sánh Raptix với các hệ thống red team tự động hiện có, và (ii) làm khung non-regression cho quá trình phát triển, đảm bảo mỗi thay đổi kiến trúc thực sự mang lại cải tiến chứ không chỉ dịch chuyển chi phí từ trục này sang trục khác. Đây là điểm khác biệt căn bản so với cách đánh giá truyền thống chỉ dựa trên solve rate.

Động cơ thiết kế Nếu chỉ đo "hệ thống tìm được bao nhiêu lỗ hổng" hoặc "chiếm được bao nhiêu host", một agent có thể đạt điểm cao bằng cách gọi rất nhiều tool và tiêu thụ một lượng lớn token, mà không thực sự thông minh hơn. Ngược lại, nếu chỉ đo chi phí, hệ thống có thể tối ưu bằng cách không làm gì cả. Vì vậy, mỗi run benchmark cần thu đồng thời cả *outcome* (kết quả đạt được) và *cost* (chi phí bỏ ra), đồng thời chạy lặp nhiều lần để đo độ ổn định. Ba trục Effectiveness, Efficiency và Reliability chính là cơ sở để cân bằng hai khía cạnh này.

Ba trục đánh giá chính

Bảng 3: Ba trục đánh giá chính và chỉ số đo lường

Trục	**Câu hỏi trung tâm**	**Chỉ số đo lường**
Effectiveness	Hệ thống có đạt được mục tiêu không?	Task success rate, số verified findings, số flag/objective đạt được, mức độ hoàn thành attack-chain.
Efficiency	Hệ thống đạt mục tiêu với chi phí bao nhiêu?	Token tiêu thụ, số model calls, số tool calls, wall-clock time, số vòng retry, coordination overhead.
Reliability	Kết quả có lặp lại được không?	Tỷ lệ thành công qua nhiều run, variance, khả năng recovery sau timeout/failure, state consistency.

Ba trục này có quan hệ đánh đổi với nhau. Một thay đổi kiến trúc có thể tăng Effectiveness nhưng giảm Efficiency (ví dụ: thêm nhiều agent con để khám phá song song), hoặc tăng Reliability nhưng giảm Efficiency (ví dụ: thêm verifier độc lập cho mọi finding). Vì vậy, một thay đổi chỉ được coi là cải tiến toàn diện khi *không* làm suy giảm nghiêm trọng hai trục còn lại.

Các bộ benchmark sử dụng

Raptix được đánh giá trên năm bộ benchmark bao phủ các nhóm capability khác nhau, từ web exploitation tới Active Directory và long-horizon operations:

Bảng 4: Các bộ benchmark và capability mục tiêu

Benchmark	**Capability chính**	**Vai trò trong suite**
WebExploitBench	Web exploitation trên target cô lập	Đo khả năng tìm và xác minh lỗ hổng web thực tế (phù hợp pha Initial Foothold).
PostExploitBench	Multi-host, post-exploitation, pivot	Đo mission nhiều bước và phụ thuộc trạng thái (phù hợp pha Network Propagation).
GOAD [6]	Windows / Active Directory	Đo enumeration, privilege path và lateral movement trong AD lab.
Nilgiri [7]	Long-horizon cyber range	Đo khả năng duy trì kế hoạch và state qua chuỗi nhiệm vụ dài.
InfraGauntlet [8]	Linux/network, non-HTTP services	Bổ sung pure infrastructure, false-positive control và pivoting.

Việc chọn nhiều bộ benchmark khác nhau nhằm tránh thiên lệch về một nhóm lỗ hổng dễ đo như web injection. Một hệ thống có thể đạt điểm cao trên WebExploitBench nhưng thất bại hoàn toàn trên GOAD hoặc Nilgiri nếu kiến trúc của nó không được thiết kế cho mission dài hoặc AD attack. Do đó, việc đánh giá Raptix cần được thực hiện trên *toàn bộ* suite, không chỉ trên một bộ đơn lẻ.

Chỉ số bổ trợ cho mô hình hai pha

Vì Raptix dùng mô hình hai pha, benchmark ghi thêm: (i) **thời điểm chuyển pha** (số bước/tool call trước khi chuyển từ Initial Foothold sang Network Propagation); (ii) **số attack path đã thử trong AKG**; (iii) **tỷ lệ credential reuse thành công** (đo hiệu quả truy vấn `CAN_AUTH_AS` và `PIVOT_VIA`); (iv) **số nhánh đã loại trừ** (đo cơ chế tránh lặp hành động).

Pin baseline và quy trình non-regression

Trước mỗi vòng tối ưu, cần pin: commit hash, model/provider/temperature, toolset (tên + version), cấu hình benchmark và giới hạn tài nguyên. Sau đó chạy baseline ít nhất 5 lần, áp dụng thay đổi, chạy lại cùng điều kiện. Do quy mô pilot nhỏ (5 run), kết quả được phân tích theo phương pháp mô tả: tỷ lệ thành công với Wilson interval cho chỉ số nhị phân; median và khoảng dao động cho chỉ số liên tục. Kiểm định suy diễn chỉ áp dụng khi $n \geq 20$. Thay đổi chỉ được giữ khi cải thiện ít nhất một trục mà không làm suy giảm nghiêm trọng hai trục còn lại.

3.4 Liên hệ và so sánh với các cách đã có

Sau khi trình bày kiến trúc và cách giải quyết đề xuất, phần này đặt Raptix trong tương quan với các hệ thống đã khảo sát ở phần 3.1, đồng thời so sánh với hai cách tiếp cận phổ biến khác trong lĩnh vực tự động hóa an toàn thông tin: prompt-only LLM và scanner truyền thống. Mục tiêu của phần này không phải là khẳng định Raptix vượt trội hơn tất cả, mà là làm rõ *đóng góp cụ thể* của thiết kế đề xuất so với từng nhóm giải pháp đã có.

Kế thừa và điều chỉnh từ các hệ thống tham chiếu

Raptix kế thừa kiến trúc multi-agent theo kill chain và sandbox biệt lập của **Decepticon** [4] (bổ sung AKG hợp nhất và cơ chế chuyển pha tường minh), tiếp thu nguyên tắc tách planning/execution và auxiliary services của **Incalmo** [5] (mở rộng thành hai pha và bổ sung verifier độc lập), và học hỏi pattern verification-centric từ **Pentest-AI** [3] (mở rộng sang nhiều loại target, tích hợp vào finding lifecycle). CyberStrikeAI [1] và Strix [2] cung cấp pattern về MCP/tool registry và skill-as-document.

So sánh với prompt-only LLM

Cách tiếp cận prompt-only (cấp quyền shell cho LLM rồi để tự quyết định) triển khai nhanh nhưng có ba hạn chế căn bản: (i) thiếu state persistence — sau nhiều bước, thông tin cũ bị "trôi dạt", agent lặp lại hành động hoặc giả định sai; (ii) không phân biệt fact với hypothesis — không có verifier độc lập; (iii) không tái lập được — cùng prompt cho kết quả rất khác nhau giữa các run. Raptix giải quyết ba điểm này bằng AKG (nguồn sự thật duy nhất), finding lifecycle với verifier, và quy trình benchmark có pin baseline.

Đóng góp chính của hướng thiết kế

Ba đóng góp chính của Raptix: (i) **AKG hợp nhất** — thay vì tách findings khỏi workspace (Decepticon [4]) hoặc tách environment state khỏi attack graph (Incalmo [5]), AKG Raptix hợp nhất tất cả thực thể vào một đồ thị duy nhất; (ii) **mô hình hai pha với chuyển pha tường minh** — kết hợp phân chia sub-agent theo pha của Decepticon [4] với declarative task của Incalmo [5], bổ sung cơ chế chuyển pha dựa trên state AKG; (iii) **verification tích hợp vào finding lifecycle** — mở rộng verifier/oracle của Pentest-AI [3] từ web sang nhiều loại target, tích hợp trực tiếp vào AKG (candidate → validated → verified → reported).

4 Mô tả phần mềm cài đặt

4.1 Môi trường phát triển và tổ chức mã nguồn

Môi trường nền tảng

Prototype Raptix kế thừa Decepticon, do đó yêu cầu Docker Compose v2 (để chạy sandbox Kali Linux, Neo4j và orchestrator runtime), ít nhất 16 GB RAM, và hỗ trợ macOS/Linux/Windows WSL2. Hai mạng Docker biệt lập (`sandbox-net` và `decepticon-net`) được tạo tự động để ngăn cách mặt phẳng thực thi và mặt phẳng quản lý.

Tổ chức mã nguồn

Mã nguồn Raptix được quản lý bằng Git và tổ chức theo các nhóm module tương ứng với năm lớp chức năng đã mô tả ở phần 3.2.1:

Bảng 5: Tổ chức mã nguồn Raptix theo nhóm module

Nhóm module	**Thư mục chính**	**Trách nhiệm**
Mission & Policy	`mission/`, `policy/`	Nhận mission, kiểm tra scope và policy, nạp thông tin ban đầu vào AKG.
Planner & Orchestrator	`planner/`, `orchestrator/`	Phân rã nhiệm vụ, chọn chiến lược khai thác theo pha, điều phối sub-agent.
Specialist Agent	`agents/`	Các sub-agent chuyên biệt theo domain (recon, exploit, post-exploit, AD, cloud).
Tool Executor & Sandbox	`executor/`, `sandbox/`	Thực thi lệnh trong sandbox, quản lý tmux session, enforce scope guard.
Knowledge & Verification	`akg/`, `verifier/`, `reporting/`	AKG, event stream, verifier độc lập và reporting engine.
Benchmark Harness	`benchmark/`	Chạy benchmark, thu metric, so sánh baseline và thay đổi.

Mã nguồn được phát triển theo nhánh làm việc riêng (`feature/akg`, `feature/two-phase`, v.v.), mỗi nhánh tương ứng với một thay đổi kiến trúc cụ thể. Trước khi thực hiện vòng tối ưu, baseline được pin theo commit đã được kiểm tra, đảm bảo mọi so sánh về sau đều dựa trên cùng một điểm xuất phát.

4.2 Các artifact phục vụ phát triển

Ngoài mã nguồn, Raptix duy trì một tập artifact đóng vai trò contract cho cả con người và coding agent: `agent/comparative.md` (ghi lại quyết định thiết kế), `agent/akg-schema.json` (schema machine-readable cho AKG), `agent/two-phase-policy.md` (điều kiện chuyển pha),

`agent/workspace.dsl` (mô hình kiến trúc C4), và benchmark config + raw artifacts (pin môi trường cho non-regression). Nguyên tắc là mọi artifact phải đủ ngắn để luôn được đọc, đủ chính xác để phát hiện drift khi implementation thay đổi.

4.3 Quy trình chạy một vòng thử nghiệm

Quy trình chạy một vòng thử nghiệm của Raptix được chuẩn hóa thành sáu bước, nhằm tách biệt việc "code chạy được" khỏi việc "thay đổi làm hệ thống tốt hơn". Một commit chỉ được coi là cải tiến khi có bằng chứng trên benchmark hoặc giải quyết được failure mode đã quan sát mà không tạo regression đáng kể ở nhóm capability khác.

Sáu bước của một vòng thử nghiệm

1. Chọn commit và kiểm tra môi trường (model, provider, toolset đúng với cấu hình đã pin).
2. Khởi tạo target lab cô lập (Docker/VM); nạp scope vào AKG qua lớp Mission & Policy.
3. Chạy agent; thu model call log, tool call log, event stream, evidence artifacts và thời điểm chuyển pha.
4. Chấm điểm bằng verifier; kiểm tra mỗi finding đều có evidence và đạt trạng thái `verified` trong AKG.
5. So sánh với baseline theo ba trục; phân tích failure mode để xác định lớp cần cải thiện.
6. Quyết định giữ/sửa/rollback; cập nhật tài liệu kiến trúc và schema nếu giữ.

5 Kết quả đạt được và hướng phát triển

5.1 Kết quả và kỹ năng thu được

Kết quả nghiên cứu và thiết kế

Kết quả lớn nhất của kỳ thực tập là xây dựng được cách tiếp cận có hệ thống cho bài toán AI Agent Red Team, thay vì bắt đầu bằng việc thêm prompt hoặc tool một cách rời rạc. Em đã hoàn thành khảo sát hai hệ thống red team tiên tiến là Decepticon và Incalmo, đồng thời tham chiếu các hệ thống pentest mã nguồn mở (CyberStrikeAI, Strix, Pentest-AI) để rút ra các pattern kiến trúc có giá trị. Trên cơ sở đó, em xác định hướng phát triển prototype **Raptix** kế thừa Decepticon làm nền tảng.

Các kết quả kỹ thuật cụ thể bao gồm:

- **Kiến trúc tổng thể Raptix:** xác định năm lớp chức năng (Mission & Policy, Planner & Orchestrator, Specialist Agent, Tool Executor & Sandbox, Knowledge & Verification) với ranh giới trách nhiệm rõ ràng và sáu nguyên tắc thiết kế xuyên suốt.
- **Attack Knowledge Graph (AKG):** thiết kế đồ thị hợp nhất với bảy loại node (Asset, Identity, Credential, Vulnerability, Finding, Action, Evidence), mười ba loại cạnh chính, ba invariant (schema, provenance, scope), và cơ chế versioning dựa trên event stream.
- **Mô hình khai thác hai pha:** phân tách quy trình khai thác thành pha Initial Foothold (breadth-first, ưu tiên độ phủ) và pha Network Propagation (depth-first, ưu tiên đường tới mục tiêu), với cơ chế chuyển pha tường minh dựa trên trạng thái của AKG.
- **Attack vector reasoning trên AKG:** xác định năm loại suy luận chính (truy vấn đường đi, phát hiện credential reuse path, phát hiện privilege escalation path, loại trừ nhánh đã thất bại, ưu tiên hóa theo chi phí và rủi ro OPSEC).
- **Khung benchmark ba trục:** thiết kế khung đánh giá theo Effectiveness, Efficiency và Reliability, kèm theo các chỉ số bổ trợ cho mô hình hai pha (thời điểm chuyển pha, số attack path đã thử, tỷ lệ credential reuse thành công).
- **Quy trình phát triển có kỷ luật:** chuẩn hóa quy trình sáu bước cho một vòng thử nghiệm, bao gồm pin baseline, chạy lặp, so sánh theo ba trục và phân tích failure mode.

Kỹ năng và kiến thức thu nhận được

Kỳ thực tập giúp em thu nhận bốn nhóm kỹ năng: (i) phân tích codebase lớn để rút ra pattern kiến trúc tái sử dụng; (ii) thiết kế AI Agent với multi-agent orchestration và graph-based state; (iii) đánh giá định lượng với baseline pin và phân loại failure mode; (iv) quy trình red team thực chiến và tư duy an toàn (scope, authorization, audit, reproducibility). Quan trọng hơn là chuyển biến tư duy từ prompt engineering sang system engineering, và từ tối ưu cảm tính sang tối ưu có kiểm soát theo ba trục.

5.2 Hạn chế và hướng phát triển

Hạn chế hiện tại

Năm hạn chế chính cần giải quyết trong giai đoạn tiếp theo: (i) **benchmark thiếu số liệu chạy lặp** — khung đã thiết kế xong nhưng chưa có đủ run thống kê, đặc biệt trên GOAD và Nilgiri; (ii) **verifier chưa chuẩn hóa** — web finding kiểm tra được bằng HTTP control, nhưng privilege escalation và lateral movement cần verifier theo lớp (deterministic → heuristic → human review); (iii) **multi-agent chưa chứng minh lợi thế** — coordination overhead chưa được đo, chưa có ngưỡng task complexity cho delegation; (iv) **AKG mới ở mức thiết kế** — Neo4j kế thừa từ Decepticon làm prototype backend, schema hợp nhất mới chưa được triển khai đầy đủ, chưa có transaction cho ghi song song; (v) **cơ chế chuyển pha chưa được kiểm chứng thực nghiệm** — ngưỡng tối ưu, chiến lược quay lại pha 1 và cách đánh giá quality của attack path đề xuất đều còn là câu hỏi mở.

Hướng phát triển tiếp theo

Bốn hướng phát triển ưu tiên: (i) hoàn thiện schema AKG và triển khai đầy đủ trên Neo4j (xây transaction cho ghi song song, migration từ schema Decepticon sang AKG hợp nhất); (ii) xây verifier theo lớp (deterministic → heuristic → human review, tích hợp vào finding lifecycle); (iii) kiểm chứng thực nghiệm cơ chế chuyển pha và ngưỡng tối ưu; (iv) mở rộng benchmark sang GOAD, Nilgiri và InfraGauntlet để kiểm chứng mô hình hai pha trong mission dài đa host.

Tài liệu

[1] CyberStrikeAI Contributors. CyberStrikeAI: AI-powered Penetration Testing with MCP Tool Registry. GitHub repository, 2024.

[2] Strix Contributors. Strix: Autonomous Pentesting Agent with Sandbox and Graph of Agents. GitHub repository, 2024.

[3] Pentest-AI Contributors. Pentest-AI: Verification-centric Automated Pentesting with Oracle and Scope Guard. GitHub repository, 2024.

[4] BitterSecurity. Decepticon: Professional AI Red Team Agent with Multi-agent Kill Chain and Neo4j Knowledge Graph. GitHub repository, `https://github.com/BitterSecurity/decepticon`, 2025. Tự báo cáo 102/104 (98.08%) trên XBOW Validation Benchmark.

[5] Keane Lucas, Matthew Singer, et al. Incalmo: An Autonomous LLM-assisted System for Red Teaming Multi-Host Networks. arXiv:2501.16466, 2025. Chấp thuận tại IEEE S&P 2026. MHBench: 37/40 vs. 3/40 SOTA.

[6] Mayfly272. GOAD – Game of Active Directory: A Pentest Active Directory Lab. GitHub repository, 2022.

[7] SPARC Research Group. Nilgiri: Long-Horizon Cyber Range for Autonomous Security Agents. GitHub repository, 2025.

[8] InfraGauntlet Contributors. InfraGauntlet: Linux/Network Benchmark for AI Red Team Agents. GitHub repository, 2025.